DEPLOYMENT RECORD · NESLIFE CONSULTATION

# Getting the app onto consult.aaryad.com

The subdomain now resolves, serves HTTPS, and reaches both backend services. Two API routes the app depends on were never built — that is the one thing still standing between this and a working patient flow.

**Date** 1 September 2026 **Domain** consult.aaryad.com **Host** 103.127.31.153

**Machine** neslifeconsult-prod

---

| | | |
|---|---|---|
| DNS<br>● **Resolving**<br>A record live on every public resolver, TTL 300. | TLS<br>● **Valid**<br>Let's Encrypt, expires 29 Nov 2026, auto-renewing. | NESTJS :4100<br>● **Reachable**<br>Admin routes answer, guards enforcing. |
| EXPRESS :4000<br>● **Reachable**<br>OTP issues codes, call signalling responds. | PATIENT PROFILE<br>● **Missing**<br>Four routes return 404 — never implemented. | DOCTOR ONBOARDING<br>● **Missing**<br>Three routes return 404 — never implemented. |

## 01 Three companies, not one

The confusion at the start was reasonable: everyone says "hosting" for three different things, and here they are genuinely three different vendors.

WHO CONTROLS WHAT

| LAYER | VENDOR | WHAT IT DOES | LOGIN NEEDED? |
|---|---|---|---|
| **Domain & DNS** | Hostinger (hPanel) | Maps names to IP addresses. Nothing else. | Yes — this is where the record was added |
| **The server** | Utho / MicroHost, Noida | The Ubuntu VM that actually runs the code | No — SSH is enough |
| **The database** | MongoDB Atlas | Stores users, doctors, OTP requests | Only to change the IP allowlist |

The server IP was identified by reverse DNS and a registry lookup, which is how we knew Hostinger was not involved:

IDENTIFYING THE SERVER'S OWNER

```
$ nslookup 103.127.31.153
   → 10312731153.network.microhost.in

$ whois 103.127.31.153
   → MicroHost / Utho, A-43 Sector 63, Noida
   → noc@utho.com
```

> WHY THIS MATTERS LATER
>
> Hostinger's browser terminal only connects to Hostinger's own machines. Every server command in this document has to run over SSH to the Utho box — never in hPanel.

## 02 Pointing the name at the server

The message from Bhavesh said the subdomain was already *created*. It was not — at least not in DNS. A check across eight likely names found nothing, so the record had to be added by hand.

The zone is authoritative at Hostinger, confirmed by its nameservers:

CONFIRMING WHO RUNS THE ZONE

```
$ nslookup -type=NS aaryad.com
   athena.dns-parking.com    ← Hostinger
   apollo.dns-parking.com    ← Hostinger
```

One record was added. Nothing existing was touched — which mattered, because the root `A` record points at Hostinger's own web hosting and editing it would have taken the main site down.

AARYAD.COM ZONE, AFTER THE CHANGE

| TYPE | NAME | CONTENT | TTL | STATUS |
|---|---|---|---|---|
| CNAME | www | aaryad.com | 300 | untouched |
| A | @ | 72.61.169.24 | 14400 | untouched |
| A | **consult** | **103.127.31.153** | 300 | added |

### How to add a record in hPanel

1. **Domains → DNS / Nameservers** for aaryad.com
2. Scroll to **Manage DNS records**
3. Type `A`, Name `consult` — *not* the full domain, Hostinger appends `.aaryad.com` itself
4. Value `103.127.31.153`, TTL `300`
5. **Add Record**

It went live on public resolvers in **80 seconds**.

> NEVER PRESS THIS
>
> **Reset DNS records** sits directly below the record table and wipes the whole zone back to defaults. It would take down aaryad.com, the www alias, and any mail routing along with it.

### Why TTL matters

TTL is how long the rest of the internet is allowed to remember an answer. It was set to **300 seconds** deliberately: if the IP is ever wrong, a correction spreads in five minutes rather than four hours. Raise it back to 14400 once things settle.

## 03 The HTTPS certificate

DNS alone was not enough. Before the certificate existed, port 443 refused every connection — and Android would have blocked plain `http://` anyway, because the app's network policy forbids cleartext to any host but the loopback aliases.

nginx was carrying a catch-all `server_name _;`. Certbot needs a server block that actually claims the domain, so that was set first, with a backup taken beforehand.

```
ON THE SERVER, AS ROOT


# 1 — back up, then claim the domain
cp /etc/nginx/sites-available/consultation /root/consultation.nginx.bak
sed -i 's/^\(\s*\)server_name .*/\1server_name consult.aaryad.com;/' \
    /etc/nginx/sites-available/consultation
nginx -t                       → test is successful
systemctl reload nginx

# 2 — issue and install the certificate
certbot --nginx -d consult.aaryad.com
    email .................. wealthcondigital@gmail.com
    terms of service ....... Y
    EFF mailing list ....... N
                               → Successfully deployed certificate
```

Verified independently from outside the server, rather than trusting the success message:

```
EXTERNAL VERIFICATION


subject   CN = consult.aaryad.com
issuer    Let's Encrypt (CN = YE1)
valid     31 Aug 2026  →  29 Nov 2026
http://   301 redirect → https://
```

WHAT CERTBOT PRODUCED

| ITEM | WHERE / WHAT |
|---|---|
| Public certificate | `/etc/letsencrypt/live/consult.aaryad.com/fullchain.pem` |
| Private key | `/etc/letsencrypt/live/consult.aaryad.com/privkey.pem` |
| nginx changes | Added the `listen 443 ssl` block and an HTTP→HTTPS redirect |
| Renewal | A systemd timer renews at roughly 60 days — no action needed |

TWO THINGS WORTH KNOWING

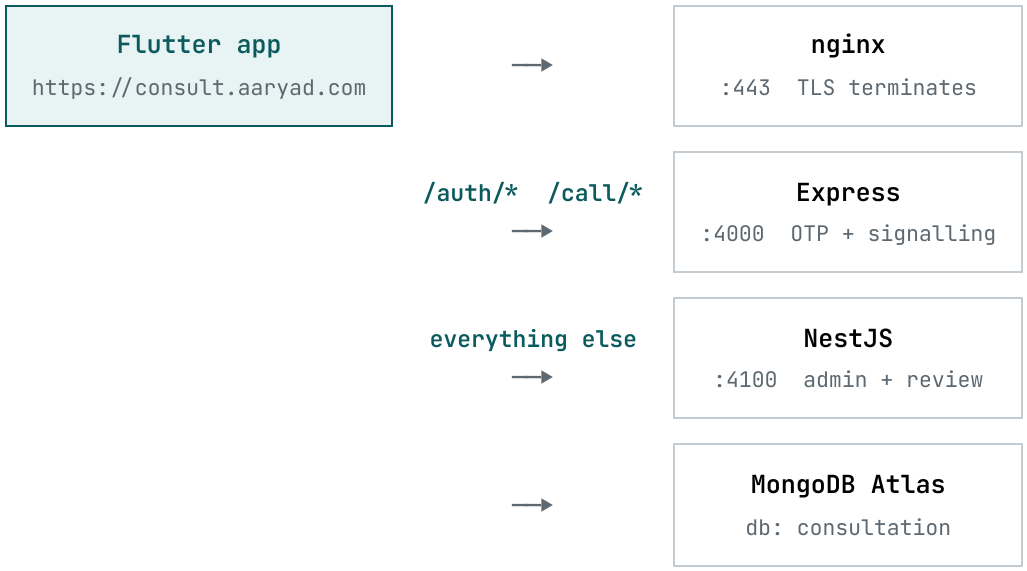
A 90-day expiry is normal for Let's Encrypt, not a trial. And `privkey.pem` should never be copied off the machine — it is the one file that makes the certificate meaningful.

Run `certbot renew --dry-run` once to prove auto-renewal works, rather than discovering it is broken in two months.

---

## 04 How a request reaches the code

There are two Node services, and the app is deliberately never told. nginx presents one hostname and splits traffic by path, so the app holds a single base URL with no port number.




Both Node processes bind to localhost only. nginx is the sole public listener, which is why the whole system needed exactly one certificate.

They run under pm2 in **fork** mode, one instance each, on purpose: the Express service keeps WebRTC session data in per-process memory, so a second worker would answer half the polls from an empty store and calls would silently fail to connect.

## 05 What works, tested over HTTPS

Every endpoint the app calls was probed against the live hostname. A `401` here is a *good* result — it proves the request reached the service, the database is connected, and the auth guard is running.

LIVE PROBE RESULTS

| ENDPOINT | SERVICE | CODE | READING |
|---|---|---|---|
| `GET /health` | NestJS | `200` | Up |
| `POST /auth/otp/request` | Express | `202` | Returns a verification id — OTP works |
| `POST /auth/otp/verify` | Express | `400` | Correct validation error |
| `POST /auth/signout` | Express | `401` | Guard active |
| `GET /call/sdp/:channel` | Express | `200` | Signalling responds |
| `GET /admin/me` | NestJS | `401` | Guard active |
| `GET /admin/doctors` | NestJS | `401` | Guard active |
| `GET /patients/me` | NestJS | `404` | Route does not exist |
| `GET /patients/me/exists` | NestJS | `404` | Route does not exist |
| `GET /doctors/me/application` | NestJS | `404` | Route does not exist |
| `GET /doctors/me/practice-setup` | NestJS | `404` | Route does not exist |
| `GET /doctors/me/availability` | NestJS | `404` | Route does not exist |

## 06 The remaining blocker

> NOT A CONFIGURATION PROBLEM
>
> Those seven `404`s are not nginx, not DNS, and not TLS. The route handlers were never written. Searching both backend codebases for `patients`, `doctors/me`, `practice-setup` and `availability` returns nothing at all.

The NestJS application registers exactly one feature module:

CONSULTATION_API/API/SRC/APP.MODULE.TS

```
imports: [ ConfigModule, MongooseModule, AdminModule ]
controllers: [ HealthController ]

// Everything it serves:
   /admin/*      admin login, OTP, review queue
   /health

// PatientsModule — does not exist
// DoctorsModule  — does not exist
```

The project's own data model document already flagged this. Line 117 reads: "*`doctor_fees`, `doctor_availability` — to build.*" It was known work, not a regression.

### What a user actually experiences

The app hardcodes the live backend at `main.dart:21`, so there is no fallback to sample data for these screens. A patient will:

1. Enter their phone number — `works`
2. Receive and enter the OTP — `works`
3. Land on the profile step, which calls `/patients/me/exists` — `404`
4. See *"Something went wrong"* and get no further

Doctor onboarding fails the same way. Admin review and video calling are unaffected.

## Which screens use live data at all

DATA SOURCE WIRING IN CORE/DI/INJECTOR.DART

| FEATURE | SOURCE | RESULT |
|---|---|---|
| Auth / OTP | Live API | `working` |
| Admin queue | Live API | `working` |
| Video call | Live API | `working` |
| Patient profile | Live API | `404` |
| Doctor onboarding | Live API | `404` |
| Home, chat, doctor list, schedule, payment, matchmaking | Sample data | `not real yet` |

## 07 The change made in the app

The app previously defaulted to a laptop on the office LAN. It now defaults to the live subdomain.

LIB/CORE/CONFIG/APP_CONFIG.DART

```
- defaultValue: 'http://192.168.0.158:4000'
+ defaultValue: 'https://consult.aaryad.com'
```

No port and no path — nginx handles the split. Building the app now needs no extra flags:

BUILDING

```
# production — uses the new default
flutter build apk

# local backend — override per build, no file edit
flutter run --dart-define=API_BASE_URL=http://10.0.2.2:4000
```

A BUG THIS UNCOVERED

The old default `192.168.0.158` could never have worked on Android. The app's cleartext exception list permits only `10.0.2.2`, `localhost` and `127.0.0.1`, so requests to a LAN IP were blocked by the platform before leaving the device. It worked on iOS only because the iOS config allows local networking. If Android testing had been failing mysteriously, this was why — and moving to HTTPS resolves it.

## 08 What is still outstanding

OPEN ITEMS, MOST BLOCKING FIRST

| ITEM | WHY IT MATTERS | PRIORITY |
|---|---|---|
| **Build PatientsModule + DoctorsModule** | Seven routes. Without them a patient cannot get past login. | `blocking` |
| **Rotate the shared credentials** | The root password, SMS gateway key and Mongo password were all sent over WhatsApp in plain text. | `urgent` |
| **SSH keys, then disable password login** | Removes the shared-password problem permanently. | `soon` |
| **Investigate consultation-auth restarts** | 36 restarts against 3 for the API. Usually the Atlas IP allowlist. | `soon` |
| **Replace remaining sample data** | Home, chat, doctor list, schedule and payment still show fixtures. | `before launch` |
| **Retire the Express service** | Moving OTP into NestJS removes the split, the second env file and the extra nginx block. | `later` |
| **Document storage bucket** | `DOCUMENTS_BUCKET` is unset, so document links are placeholders. | `later` |

09

# Quick reference

```
CONNECTING AND CHECKING HEALTH


# from Windows Terminal / PowerShell — not Hostinger
ssh root@103.127.31.153

pm2 list                            # both services should say online
pm2 logs consultation-api           # NestJS
pm2 logs consultation-auth          # Express
bash /srv/consultation/verify.sh    # exercises every endpoint

curl -I https://consult.aaryad.com/health
certbot certificates                # expiry dates
certbot renew --dry-run             # prove renewal works
```

WHERE THINGS LIVE

| THING | LOCATION |
|---|---|
| Application code | `/srv/consultation/{api,server}` |
| nginx site config | `/etc/nginx/sites-available/consultation` |
| Pre-TLS nginx backup | `/root/consultation.nginx.bak` |
| Certificates | `/etc/letsencrypt/live/consult.aaryad.com/` |
| Application logs | `/var/log/consultation/*.log` |
| nginx logs | `/var/log/nginx/consultation.*.log` |
| Release backups | `/srv/backups/` |

### Known traps on this machine

- **DHCP hands out no DNS servers.** If `apt` or `npm` fails to resolve anything, check `/etc/systemd/resolved.conf` first.
- **No IPv6 route.** apt is forced to IPv4 via `/etc/apt/apt.conf.d/99force-ipv4`.
- **Atlas allowlist.** If everything returns 502, check Network Access in Atlas before reading any code — neither service can serve a single request without the database.
- **Let's Encrypt rate limit.** Five certificates per domain per week. If certbot fails on DNS, wait and retry rather than looping.

Neslife Consultation — deployment record, 1 September 2026
All endpoint results in this document were verified against the live host, not inferred from configuration.